topic_3_component_based_ecs

Theory, Edge Cases & Code Examples

- ✓ What is Component-Based Architecture?
- ✓ Simple Explanation: LEGO Blocks Analogy
- ✓ Why Use Component-Based Architecture?
- ✓ Problem with Deep Inheritance
- ✓ Solution: Composition

→ Swipe to learn →

topic_3_component_based_ecs

Theory

📌 What is Component-Based Architecture?

Component-Based Architecture is a design pattern where you build objects by **composing** them from smaller, reusable pieces (components) rather than using inheritance hierarchies.

Entity-Component-System (ECS) is a specific implementation of component-based architecture that separates:
- **Entities**: Unique IDs (like "Player", "Enemy", "Car")
- **Components**: Pure data (like Position, Velocity, Health)
- **Systems**: Logic that operates on components (like MovementSystem, RenderSystem)

---

topic_3_component_based_ecs

Theory

📌 Simple Explanation: LEGO Blocks Analogy

Imagine building with LEGO:

Traditional Inheritance	Component-Based (ECS)
Pre-built toy models	**Individual LEGO blocks**
You buy "Car toy", "Robot toy"	You buy blocks: wheels, arms, body parts
Limited to what's pre-built	Combine any blocks to create anything
Hard to mix parts (robot with car wheels?)	Easy to mix: robot with wheels!
"IS-A" relationship	**"HAS-A" relationship**
Car IS-A Vehicle	Car HAS wheels, engine, body

Key Insight: Instead of saying "A car **IS-A** vehicle", you say "A car **HAS** position, velocity, and mesh components".

---

topic_3_component_based_ecs

Theory

📌 Why Use Component-Based Architecture?

topic_3_component_based_ecs

Theory

📌 Problem with Deep Inheritance

```
                  GameObject
                      |
          +-----------+-----------+
          |                       |
    PhysicsObject          RenderableObject
          |                       |
    +-----+-----+           +-----+-----+
    |           |           |           |
MovableObject StaticObject VisibleObject AnimatedObject
    |
FlyingObject
    |
  Player
```

Problems:

1. **Fragile**: Change one class → breaks everything below
2. **Rigid**: Can't have "visible but not physical" easily
3. **Code Duplication**: Flying logic duplicated if you need "flying enemy"
4. **Hard to Extend**: Want a "flying, animated, physics-enabled" object? Create 10 parent classes?

topic_3_component_based_ecs

Theory

📌 Solution: Composition

```
Entity ID: 1001 (Player)
Components:
  - PositionComponent {x: 10, y: 5}
  - VelocityComponent {dx: 2, dy: 0}
  - HealthComponent {hp: 100}
  - MeshComponent {model: "player.obj"}

Entity ID: 1002 (Enemy)
Components:
  - PositionComponent {x: 20, y: 5}
  - VelocityComponent {dx: -1, dy: 0}
  - HealthComponent {hp: 50}
  - AIComponent {behavior: "patrol"}
```

Benefits:
- ✅ **Flexible**: Mix any components
- ✅ **Reusable**: Same components for Player and Enemy
- ✅ **Maintainable**: Change one component doesn't affect others
- ✅ **Data-Oriented**: Components are pure data → cache-friendly

---

topic_3_component_based_ecs

Theory

📌 Core ECS Concepts

topic_3_component_based_ecs

Theory

📌 1. Entities (Just IDs)

An **entity** is just a **unique identifier** (like an integer ID). It represents "something" in your system but holds NO DATA and NO LOGIC.

```
using EntityID = uint32_t;

EntityID player = 1001;
EntityID enemy = 1002;
EntityID bullet = 1003;
```

Think of it as: A label or tag. Like a student ID number.

---

topic_3_component_based_ecs

Theory

📌 2. Components (Pure Data)

A **component** is a **plain struct** holding data. No methods, no logic.

```
struct PositionComponent {
    float x, y;
};

struct VelocityComponent {
    float dx, dy;
};

struct HealthComponent {
    int hp;
};
```

Think of it as: Attributes or properties. Like "height", "weight", "age" on a form.

Key Rule: Components should be **POD (Plain Old Data)** - simple structs with no complex methods.

---

topic_3_component_based_ecs

Theory

📌 3. Systems (Pure Logic)

A **system** contains **logic** that operates on entities with specific components.

```
class MovementSystem {
public:
    void update(float dt, ComponentManager& components) {
        // For each entity that has BOTH Position and Velocity
        for (auto entity : entitiesWithComponents<Position, Velocity>()) {
            auto& pos = components.get<Position>(entity);
            auto& vel = components.get<Velocity>(entity);

            pos.x += vel.dx * dt;
            pos.y += vel.dy * dt;
        }
    }
};
```

Think of it as: Workers who process data. Like a cashier (processes payments), chef (processes orders).

Key Rule: Systems should be **stateless** - they don't store data, they process components.

---

topic_3_component_based_ecs

Theory

📌 How ECS Works: Step-by-Step

Let's simulate a simple game frame:

```
Frame 1:
┌──────────────────────────────────────────────┐
│ 1. CREATE ENTITIES                           │
│    - Player (ID: 1)                          │
│    - Enemy (ID: 2)                           │
└──────────────────────────────────────────────┘
         ↓
┌──────────────────────────────────────────────┐
│ 2. ADD COMPONENTS                            │
│    Entity 1: Position(0,0), Velocity(1,0), HP │
│    Entity 2: Position(10,0), Velocity(-1,0), HP │
└──────────────────────────────────────────────┘
         ↓
┌──────────────────────────────────────────────┐
│ 3. SYSTEMS PROCESS                           │
│    MovementSystem → updates positions        │
│    CollisionSystem → checks if entities collide │
│    RenderSystem → draws entities on screen   │
└──────────────────────────────────────────────┘
         ↓
Frame 2: Repeat step 3
```

---

topic_3_component_based_ecs

Theory

📌 ECS vs Traditional OOP

Traditional OOP	Entity-Component-System
Inheritance hierarchy	**Composition**
`Player extends GameObject`	`Player = {Position, Velocity, Health}`
Data + Logic together	**Data separate from Logic**
`class Player { int hp; void takeDamage() }`	`Component: {int hp}` + `System: processDamage()`
Hard to add behaviors	**Easy to add/remove components**
Need to change class hierarchy	Just add/remove components
Cache unfriendly	**Cache friendly**
Objects scattered in memory	Components packed in arrays
Tight coupling	**Loose coupling**
Systems depend on each other	Systems are independent

---

topic_3_component_based_ecs

Theory

📌 Data-Oriented Design Benefits

ECS is **data-oriented** rather than object-oriented:

```
// ❌ Object-Oriented (cache unfriendly)
class GameObject {
    Position pos;
    Velocity vel;
    Health hp;
    Mesh mesh;
    AI ai;
    // ... many more fields
};

std::vector<GameObject*> objects;  // Scattered in memory

for (auto obj : objects) {
    obj->update();  // Cache misses! Each object far apart in memory
}

// ✅ Data-Oriented (cache friendly)
std::vector<Position> positions;  // All positions together
std::vector<Velocity> velocities; // All velocities together

for (size_t i = 0; i < positions.size(); i++) {
    positions[i].x += velocities[i].dx;  // Sequential memory access!
}
```

Why This Matters:
- **Cache locality**: Data used together is stored together
- **SIMD-friendly**: Can vectorize operations
- **Multithreading**: Systems can run in parallel (no shared state)

---

topic_3_component_based_ecs

Theory

📌 When to Use ECS

Use ECS When	Don't Use ECS When
✅ Need high flexibility (add/remove behaviors)	❌ Simple, small projects
✅ Performance critical (games, simulations)	❌ Few entity types (< 5)
✅ Many entity types (100+ types)	❌ Deep business logic (banking, e-commerce)
✅ Data-driven design (load entities from files)	❌ Complex inheritance makes sense
✅ Runtime composition (mod support)	❌ Team unfamiliar with ECS

Example Use Cases:
- ✅ Game engines (Unity uses ECS)
- ✅ Physics simulations (particle systems)
- ✅ Robotics (sensor components + actuator systems)
- ✅ Entity management in MMOs

---

topic_3_component_based_ecs

Theory

📌 Common ECS Implementations

topic_3_component_based_ecs

Theory

📌 Approach 1: Component Manager (Simple)

```
class ComponentManager {
    std::map<EntityID, std::map<std::type_index, void*>> components;

public:
    template<typename T>
    void addComponent(EntityID entity, T component) {
        components[entity][typeid(T)] = new T(component);
    }

    template<typename T>
    T& getComponent(EntityID entity) {
        return *static_cast<T*>(components[entity][typeid(T)]);
    }
};
```

Pros: Simple to understand

Cons: Slow (pointer indirection, not cache-friendly)

---

topic_3_component_based_ecs

Theory

📌 Approach 2: Archetypes (Cache-Friendly)

Entities with same component combination share storage:

```
Archetype 1: [Position, Velocity, Health]
  Entity 1: {pos, vel, hp}
  Entity 2: {pos, vel, hp}

Archetype 2: [Position, Mesh]
  Entity 3: {pos, mesh}
  Entity 4: {pos, mesh}
```

Pros: Cache-friendly (same components together)

Cons: Moving entities between archetypes is expensive

---

topic_3_component_based_ecs

Theory

📌 Approach 3: Sparse Sets (Fast Iteration)

Each component type has its own sparse set:

```
template<typename T>
class SparseSet {
    std::vector<EntityID> dense;   // Packed entity IDs
    std::vector<T> components;      // Packed components
    std::vector<size_t> sparse;     // EntityID → index mapping
};
```

Pros: Fast iteration, fast add/remove

Cons: More memory (sparse array)

---

topic_3_component_based_ecs

Theory

📌 Real-World Example: Unity's DOTS

Unity's Data-Oriented Technology Stack (DOTS) uses ECS:

```
// Entity (just an ID)
Entity player = entityManager.CreateEntity();

// Add components
entityManager.AddComponent<Position>(player, new Position { x = 0, y = 0 });
entityManager.AddComponent<Velocity>(player, new Velocity { dx = 1, dy = 0 });

// System processes components
public class MovementSystem : SystemBase {
    protected override void OnUpdate() {
        Entities.ForEach((ref Position pos, in Velocity vel) => {
            pos.x += vel.dx * deltaTime;
            pos.y += vel.dy * deltaTime;
        }).Run();
    }
}
```

Result: Unity saw **4x-10x performance improvements** in some cases!

---

topic_3_component_based_ecs

Edge Cases

📌 1. Component Dependencies (1/3)

Problem: Some components depend on others. Example: `VelocityComponent` needs `PositionComponent` to be useful.

Naive Code:

```
void MovementSystem::update() {
    for (EntityID entity : entities) {
        // What if entity has Velocity but no Position?
        auto& vel = components.get<Velocity>(entity);
        auto& pos = components.get<Position>(entity);  // ❌ Crash if missing!

        pos.x += vel.dx;
    }
}
```

Why It Fails:
- Entity might have `Velocity` component but no `Position` component
- Calling `get<Position>()` on such an entity crashes

topic_3_component_based_ecs

Edge Cases

📌 1. Component Dependencies (2/3)

Solution 1: Query System (Check Before Use)

```
class ComponentManager {
public:
    template<typename... Components>
    std::vector<EntityID> getEntitiesWith() {
        std::vector<EntityID> result;
        for (auto& [entity, componentMap] : allComponents) {
            if ((hasComponent<Components>(entity) && ...)) {
                result.push_back(entity);
            }
        }
        return result;
    }
};

void MovementSystem::update() {
    // Only iterate entities that have BOTH Position AND Velocity
    for (EntityID entity : components.getEntitiesWith<Position, Velocity>()) {
        auto& pos = components.get<Position>(entity);
        auto& vel = components.get<Velocity>(entity);

        pos.x += vel.dx;  // ✅ Safe! We know both exist
    }
}
```

topic_3_component_based_ecs

Edge Cases

📌 1. Component Dependencies (3/3)

Solution 2: Component Groups (Pre-filtered)

```
class ComponentGroup {
    std::vector<EntityID> entities;  // Pre-filtered entities

public:
    void addEntity(EntityID entity) {
        entities.push_back(entity);
    }

    void removeEntity(EntityID entity) {
        entities.erase(std::remove(entities.begin(), entities.end(), entity));
    }
};

// Create group for entities with Position + Velocity
ComponentGroup movableGroup;

// When adding components, update groups
void addComponent(EntityID entity, VelocityComponent vel) {
    components[entity] = vel;
    if (hasComponent<Position>(entity)) {
        movableGroup.addEntity(entity);  // Add to group
    }
}

void MovementSystem::update() {
    for (EntityID entity : movableGroup.entities) {
        // All entities guaranteed to have Position + Velocity
        auto& pos = components.get<Position>(entity);
        auto& vel = components.get<Velocity>(entity);
        pos.x += vel.dx;
    }
}
```

Lesson: Always validate component existence OR use query/group systems to pre-filter.

topic_3_component_based_ecs

Edge Cases

📌 2. System Execution Order (1/4)

Problem: Systems may depend on each other's results. Example: `CollisionSystem` needs updated positions from `MovementSystem`.

Naive Code:

```
void gameLoop() {
    collisionSystem.update();  // ❌ Uses old positions!
    movementSystem.update();   // Updates positions AFTER collision check
}
```

Why It Fails:
- Collision system checks collisions using **last frame's positions**
- Movement happens **after** collision, so entities might be inside each other!

Example Scenario:

```
Frame 1:
  Player at x=5, moving right (velocity=1)
  Wall at x=10

  1. CollisionSystem: Check if player at x=5 hits wall → NO
  2. MovementSystem: Move player to x=6

Frame 2:
  Player at x=6, moving right

  1. CollisionSystem: Check if player at x=6 hits wall → NO
  ... continues until player at x=11 (INSIDE wall!)
```

topic_3_component_based_ecs

Edge Cases

📌 2. System Execution Order (2/4)

Solution 1: Explicit System Order

```
class SystemManager {
    std::vector<System*> systems;

public:
    void addSystem(System* system, int priority) {
        systems.push_back(system);
        std::sort(systems.begin(), systems.end(), [](System* a, System* b) {
            return a->priority < b->priority;
        });
    }

    void update() {
        for (auto* system : systems) {
            system->update();
        }
    }
};

// Usage
systemManager.addSystem(&movementSystem, 100);     // Run first
systemManager.addSystem(&collisionSystem, 200);    // Run after movement
systemManager.addSystem(&renderSystem, 300);       // Run last
```

📌 2. System Execution Order (3/4)

Solution 2: System Dependencies (Graph)

```
class System {
public:
    std::vector<System*> dependencies;

    virtual void update() = 0;
};

class SystemScheduler {
public:
    void scheduleSystem(System* system) {
        // Topological sort based on dependencies
        // Ensures dependencies run before dependents
    }
};

// Usage
movementSystem.dependencies = {};  // No dependencies
collisionSystem.dependencies = {&movementSystem};  // Depends on movement
renderSystem.dependencies = {&movementSystem, &collisionSystem};

scheduler.scheduleSystem(&renderSystem);  // Auto-schedules dependencies
```

Solution 3: Multi-Phase Updates

```
void gameLoop() {
    // Phase 1: Update physics
    movementSystem.update();
    gravitySystem.update();

    // Phase 2: Resolve collisions
    collisionSystem.update();

    // Phase 3: Update game logic
    aiSystem.update();

    // Phase 4: Render
    renderSystem.update();
}
```

📌 2. System Execution Order (4/4)

Lesson: Explicit system ordering is crucial. Document dependencies clearly.

---

topic_3_component_based_ecs

Edge Cases

📌 3. Component Removal During Iteration (1/4)

Problem: Removing a component while iterating over entities can invalidate iterators.

Naive Code:

```
void DamageSystem::update() {
    for (EntityID entity : entities) {
        auto& health = components.get<Health>(entity);
        health.hp -= 10;

        if (health.hp <= 0) {
            components.remove<Health>(entity);  // ❌ Invalidates iterator!
        }
    }
}
```

Why It Fails:
- `remove()` modifies the underlying container (vector, map)
- Iterators become invalid → crash or skip entities

📌 3. Component Removal During Iteration (2/4)

Solution 1: Deferred Removal (Queue)

```
class ComponentManager {
    std::vector<std::pair<EntityID, std::type_index>> removalQueue;

public:
    void queueRemoval(EntityID entity, std::type_index type) {
        removalQueue.emplace_back(entity, type);
    }

    void processRemovals() {
        for (auto [entity, type] : removalQueue) {
            // Actually remove components
            components[type].erase(entity);
        }
        removalQueue.clear();
    }
};

void DamageSystem::update() {
    for (EntityID entity : entities) {
        auto& health = components.get<Health>(entity);
        health.hp -= 10;

        if (health.hp <= 0) {
            components.queueRemoval(entity, typeid(Health));  // ✅ Safe
        }
    }
}

void gameLoop() {
    damageSystem.update();
    components.processRemovals();  // Remove at end of frame
}
```

📌 3. Component Removal During Iteration (3/4)

Solution 2: Mark for Deletion (Flag)

```
struct HealthComponent {
    int hp;
    bool markedForDeletion = false;
};

void DamageSystem::update() {
    for (EntityID entity : entities) {
        auto& health = components.get<Health>(entity);
        if (health.markedForDeletion) continue;  // Skip deleted

        health.hp -= 10;
        if (health.hp <= 0) {
            health.markedForDeletion = true;  // ✅ Safe
        }
    }
}

void cleanup() {
    for (EntityID entity : entities) {
        auto& health = components.get<Health>(entity);
        if (health.markedForDeletion) {
            components.remove<Health>(entity);
        }
    }
}
```

📌 3. Component Removal During Iteration (4/4)

Solution 3: Reverse Iteration (For Vectors)

```
void DamageSystem::update() {
    // Iterate backwards so removal doesn't affect unprocessed entities
    for (int i = entities.size() - 1; i >= 0; i--) {
        EntityID entity = entities[i];
        auto& health = components.get<Health>(entity);

        health.hp -= 10;
        if (health.hp <= 0) {
            components.remove<Health>(entity);  // ✅ Safe (already processed)
        }
    }
}
```

Lesson: Never mutate a collection while iterating forward. Use deferred operations or reverse iteration.

---

topic_3_component_based_ecs

Edge Cases

📌 4. Entity Lifetime and Dangling References (1/3)

Problem: Destroying an entity while other systems still reference it.

Naive Code:

```
EntityID bullet = createEntity();
components.add<Position>(bullet, {10, 5});

// Later...
destroyEntity(bullet);  // Entity destroyed

// Even later...
auto& pos = components.get<Position>(bullet);  // ❌ Bullet no longer exists!
pos.x += 1;  // Crash or undefined behavior
```

Why It Fails:
- Entity ID `bullet` is reused for a new entity
- Systems may still hold references to old entity
- Accessing destroyed entity's components → crash

📌 4. Entity Lifetime and Dangling References (2/3)

Solution 1: Generation Counter (Versioned Entities)

```
struct EntityID {
    uint32_t id;         // Actual ID (0, 1, 2, ...)
    uint32_t generation; // Version (incremented on reuse)
};

class EntityManager {
    std::vector<uint32_t> generations;  // Current generation for each ID
    std::queue<uint32_t> freeIDs;

public:
    EntityID createEntity() {
        uint32_t id = freeIDs.empty() ? generations.size() : freeIDs.front();
        if (!freeIDs.empty()) freeIDs.pop();

        if (id >= generations.size()) generations.resize(id + 1, 0);

        return {id, generations[id]};
    }

    void destroyEntity(EntityID entity) {
        generations[entity.id]++;  // Increment generation
        freeIDs.push(entity.id);    // Reuse ID
    }

    bool isValid(EntityID entity) {
        return entity.generation == generations[entity.id];
    }
};

// Usage
EntityID bullet = entityManager.createEntity();  // {id: 0, gen: 0}
destroyEntity(bullet);
EntityID enemy = entityManager.createEntity();   // {id: 0, gen: 1} (reused ID!)

// Later...
if (entityManager.isValid(bullet)) {  // ✅ Returns false! (gen 0 != gen 1)
    auto& pos = components.get<Position>(bullet);
}
```

📌 4. Entity Lifetime and Dangling References (3/3)

Solution 2: Entity Destruction Events

```
class EntityManager {
    std::vector<std::function<void(EntityID)>> destructionListeners;

public:
    void onEntityDestroy(std::function<void(EntityID)> callback) {
        destructionListeners.push_back(callback);
    }

    void destroyEntity(EntityID entity) {
        for (auto& listener : destructionListeners) {
            listener(entity);  // Notify all systems
        }
        // Then actually destroy
    }
};

// System clears its references
bulletSystem.init() {
    entityManager.onEntityDestroy([this](EntityID entity) {
        activeBullets.erase(entity);  // Remove from tracking
    });
}
```

Lesson: Use versioned IDs or event systems to handle entity lifetime safely.

topic_3_component_based_ecs

Edge Cases

📌 5. Over-Granular Components (Bloat) (1/2)

Problem: Creating too many small components makes systems inefficient.

Naive Code:

```
struct XComponent { float x; };
struct YComponent { float y; };
struct ZComponent { float z; };

// To move an entity, need 3 component lookups!
auto& x = components.get<XComponent>(entity);
auto& y = components.get<YComponent>(entity);
auto& z = components.get<ZComponent>(entity);
```

Why It Fails:
- **Too many lookups**: Each `get<>()` is a hash map lookup
- **Cache unfriendly**: x, y, z stored in different places
- **Verbose**: Need 3 lines for simple position access

Solution 1: Group Related Data

```
struct PositionComponent {
    float x, y, z;  // ✅ Grouped together
};

auto& pos = components.get<Position>(entity);
pos.x += vel.dx;
pos.y += vel.dy;
pos.z += vel.dz;
```

topic_3_component_based_ecs

Edge Cases

📌 5. Over-Granular Components (Bloat) (2/2)

Solution 2: Component Granularity Guidelines

Good Granularity	Too Granular	Too Coarse
`Position {x, y, z}`	`X{x}, Y{y}, Z{z}`	`GameObject {pos, vel, mesh, ai, ...}`
`Velocity {dx, dy, dz}`	`VelocityX, VelocityY`	`Physics {pos, vel, accel, mass, ...}`
`Health {hp, maxHp}`	`HP{hp}, MaxHP{max}`	`Stats {hp, mp, str, dex, ...}`

Rule of Thumb:
- Group data that's **always used together**
- Separate data that's **used by different systems**

Example:

```
// ✅ Good: Position and Velocity separate
// (RenderSystem uses Position, MovementSystem uses both)
struct Position { float x, y, z; };
struct Velocity { float dx, dy, dz; };

// ❌ Bad: Grouped when not always used together
struct PositionAndVelocity {
    float x, y, z;
    float dx, dy, dz;  // Wastes cache when rendering (only need position)
};
```

Lesson: Balance between granularity and performance. Group coherent data, separate independent data.

topic_3_component_based_ecs

Edge Cases

📌 6. Multithreading Component Access (1/3)

Problem: Multiple systems accessing same components concurrently → data races.

Naive Code:

```
// Thread 1: MovementSystem
void MovementSystem::update() {
    auto& pos = components.get<Position>(entity);
    pos.x += vel.dx;  // ❌ Write
}

// Thread 2: RenderSystem (runs in parallel)
void RenderSystem::update() {
    auto& pos = components.get<Position>(entity);  // ❌ Read while writing!
    drawAt(pos.x, pos.y);
}
```

Why It Fails:
- **Data race**: One thread writes while another reads
- **Undefined behavior**: Could read partial update

📌 6. Multithreading Component Access (2/3)

Solution 1: System Access Patterns (Read/Write Tracking)

```
class System {
public:
    virtual std::vector<std::type_index> readComponents() = 0;
    virtual std::vector<std::type_index> writeComponents() = 0;
};

class MovementSystem : public System {
    std::vector<std::type_index> readComponents() override {
        return {typeid(Velocity)};
    }
    std::vector<std::type_index> writeComponents() override {
        return {typeid(Position)};
    }
};

class SystemScheduler {
    void scheduleParallel(std::vector<System*> systems) {
        // Analyze read/write patterns
        // Only run systems in parallel if:
        //   - No system writes what another reads
        //   - No two systems write to same component

        if (canRunInParallel(movementSystem, renderSystem)) {
            runParallel(movementSystem, renderSystem);
        } else {
            runSequential(movementSystem, renderSystem);
        }
    }
};
```

📌 6. Multithreading Component Access (3/3)

Solution 2: Read-Only vs Read-Write Systems

```
class RenderSystem {
    void update() {
        // Only READ components (no writes)
        for (EntityID entity : entities) {
            const auto& pos = components.get<Position>(entity);
            drawAt(pos.x, pos.y);
        }
    }
};

// Can run multiple read-only systems in parallel safely
scheduler.runParallel({&renderSystem, &debugDrawSystem, &aiPerceptionSystem});
```

Solution 3: System Stages (Barriers)

```
void gameLoop() {
    // Stage 1: Physics (can run in parallel)
    runParallel({&movementSystem, &gravitySystem});

    barrier();  // Wait for all to finish

    // Stage 2: Game logic (can run in parallel)
    runParallel({&collisionSystem, &aiSystem});

    barrier();

    // Stage 3: Rendering (single-threaded)
    renderSystem.update();
}
```

Lesson: Track which components each system reads/writes. Use barriers or dependency graphs for safe parallelism.

topic_3_component_based_ecs

Code Examples

📌 1. Basic ECS Framework (Simple) (1/5)

Goal: Implement minimal ECS with entities, components, and one system.

Output:

Key Concepts:
1. **Entities are just IDs** (uint32_t)
2. **Components are plain structs** (Position, Velocity)
3. **Systems query for entities with specific components**
4. **Composition**: Obstacle has Position but no Velocity → doesn't move

---

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <typeindex>
#include <memory>

// ═══════════════════════════════════════════════════════════
//  ENTITIES
// ═══════════════════════════════════════════════════════════

using EntityID = uint32_t;

class EntityManager {
    EntityID nextID = 0;

public:
    EntityID createEntity() {
        return nextID++;
    }
};

// ═══════════════════════════════════════════════════════════
//  COMPONENTS (Pure Data)
// ═══════════════════════════════════════════════════════════

struct PositionComponent {
    float x, y;
};
// ...continues on next slide
```

topic_3_component_based_ecs

Code Examples

📌 1. Basic ECS Framework (Simple) (2/5)

```
// ...continued from previous slide

struct VelocityComponent {
    float dx, dy;
};

// ═══════════════════════════════════════════════════════════
//  COMPONENT MANAGER
// ═══════════════════════════════════════════════════════════

class ComponentManager {
    // Map: ComponentType -> (Map: EntityID -> Component)
    std::unordered_map<std::type_index,
        std::unordered_map<EntityID, std::shared_ptr<void>>> components;

public:
    template<typename T>
    void addComponent(EntityID entity, T component) {
        auto typeIndex = std::type_index(typeid(T));
        components[typeIndex][entity] = std::make_shared<T>(component);

        std::cout << "Added " << typeid(T).name()
                  << " to entity " << entity << "\n";
    }

    template<typename T>
    T& getComponent(EntityID entity) {
        auto typeIndex = std::type_index(typeid(T));
        return *std::static_pointer_cast<T>(components[typeIndex][entity]);
    }

    template<typename T>
    bool hasComponent(EntityID entity) {
        auto typeIndex = std::type_index(typeid(T));
        return components[typeIndex].find(entity) != components[typeIndex].end();
    }

    template<typename... Components>
    std::vector<EntityID> getEntitiesWith() {
        std::vector<EntityID> result;

// ...continues on next slide
```

topic_3_component_based_ecs

Code Examples

📌 1. Basic ECS Framework (Simple) (3/5)

```
// ...continued from previous slide
        // Get all entity IDs that have at least one component
        std::set<EntityID> allEntities;
        for (auto& [type, entityMap] : components) {
            for (auto& [entity, comp] : entityMap) {
                allEntities.insert(entity);
            }
        }

        // Check each entity has all required components
        for (EntityID entity : allEntities) {
            if ((hasComponent<Components>(entity) && ...)) {
                result.push_back(entity);
            }
        }

        return result;
    }
};

// ═══════════════════════════════════════════════════════════════
//  SYSTEMS (Pure Logic)
// ═══════════════════════════════════════════════════════════════

class MovementSystem {
public:
    void update(ComponentManager& components, float dt) {
        std::cout << "\n[MovementSystem] Updating...\n";

        // Only process entities with BOTH Position and Velocity
        for (EntityID entity : components.getEntitiesWith<PositionComponent, VelocityComponent>()) {
            auto& pos = components.getComponent<PositionComponent>(entity);
            auto& vel = components.getComponent<VelocityComponent>(entity);

            pos.x += vel.dx * dt;
            pos.y += vel.dy * dt;

            std::cout << "  Entity " << entity
                      << " moved to (" << pos.x << ", " << pos.y << ")\n";
        }
    }
// ...continues on next slide
```

📌 1. Basic ECS Framework (Simple) (4/5)

```
// ...continued from previous slide
};

// ═══════════════════════════════════════════════════════════════
//  MAIN
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════\n";
    std::cout << "  ECS Example 1: Basic Framework\n";
    std::cout << "═══════════════════════════════════════════════\n\n";

    EntityManager entityManager;
    ComponentManager componentManager;
    MovementSystem movementSystem;

    // Create entities
    EntityID player = entityManager.createEntity();
    EntityID enemy = entityManager.createEntity();
    EntityID staticObstacle = entityManager.createEntity();

    std::cout << "\n--- Creating Entities ---\n";
    std::cout << "Player ID: " << player << "\n";
    std::cout << "Enemy ID: " << enemy << "\n";
    std::cout << "Obstacle ID: " << staticObstacle << "\n";

    // Add components
    std::cout << "\n--- Adding Components ---\n";
    componentManager.addComponent(player, PositionComponent{0, 0});
    componentManager.addComponent(player, VelocityComponent{1, 0.5f});

    componentManager.addComponent(enemy, PositionComponent{10, 10});
    componentManager.addComponent(enemy, VelocityComponent{-0.5f, 0});

    componentManager.addComponent(staticObstacle, PositionComponent{5, 5});
    // No velocity! (static object)

    // Simulate 3 frames
    std::cout << "\n--- Simulating Frames ---\n";
    for (int frame = 1; frame <= 3; frame++) {
        std::cout << "\n========== Frame " << frame << " ==========\n";
// ...continues on next slide
```

topic_3_component_based_ecs

Code Examples

📌 1. Basic ECS Framework (Simple) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
        movementSystem.update(componentManager, 1.0f);  // dt = 1.0
    }

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << "  Notice: Obstacle didn't move (no Velocity component)\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

```
["═══════════════════════════════════════════════════\n  ECS Example 1: Basic Framework\n═══════════════════════════
```

topic_3_component_based_ecs

Code Examples

📌 2. Game with Multiple Systems (1/5)

Goal: Expand to multiple systems (Movement, Collision, Health, Render).

Output:

Key Concepts:
- **Multiple systems** working together
- **System execution order** matters (Movement → Collision → Health → Render)
- **Collision detection** using components
- **Damage system** modifying Health component

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <typeindex>
#include <memory>
#include <cmath>

// (Reuse EntityManager and ComponentManager from Example 1)

// ═══════════════════════════════════════════════════════════
//  COMPONENTS
// ═══════════════════════════════════════════════════════════

struct PositionComponent {
    float x, y;
};

struct VelocityComponent {
    float dx, dy;
};

struct HealthComponent {
    int hp;
};

struct ColliderComponent {
    float radius;
};
// ...continues on next slide
```

📌 2. Game with Multiple Systems (2/5)

```
// ...continued from previous slide

// ═══════════════════════════════════════════════════════════════
//  SYSTEMS
// ═══════════════════════════════════════════════════════════════

class MovementSystem {
public:
    void update(ComponentManager& components, float dt) {
        for (EntityID entity : components.getEntitiesWith<PositionComponent, VelocityComponent>()) {
            auto& pos = components.getComponent<PositionComponent>(entity);
            auto& vel = components.getComponent<VelocityComponent>(entity);

            pos.x += vel.dx * dt;
            pos.y += vel.dy * dt;
        }
    }
};

class CollisionSystem {
public:
    void update(ComponentManager& components) {
        auto entities = components.getEntitiesWith<PositionComponent, ColliderComponent>();

        // Check all pairs
        for (size_t i = 0; i < entities.size(); i++) {
            for (size_t j = i + 1; j < entities.size(); j++) {
                EntityID e1 = entities[i];
                EntityID e2 = entities[j];

                auto& pos1 = components.getComponent<PositionComponent>(e1);
                auto& pos2 = components.getComponent<PositionComponent>(e2);
                auto& col1 = components.getComponent<ColliderComponent>(e1);
                auto& col2 = components.getComponent<ColliderComponent>(e2);

                float dx = pos2.x - pos1.x;
                float dy = pos2.y - pos1.y;
                float distance = std::sqrt(dx*dx + dy*dy);

                if (distance < col1.radius + col2.radius) {
                    std::cout << "  COLLISION: Entity " << e1
// ...continues on next slide
```

📌 2. Game with Multiple Systems (3/5)

```
// ...continued from previous slide
                            << " and Entity " << e2 << "\n";

                    // Apply damage if has health
                    if (components.hasComponent<HealthComponent>(e1)) {
                        auto& hp1 = components.getComponent<HealthComponent>(e1);
                        hp1.hp -= 10;
                    }
                    if (components.hasComponent<HealthComponent>(e2)) {
                        auto& hp2 = components.getComponent<HealthComponent>(e2);
                        hp2.hp -= 10;
                    }
                }
            }
        }
    }
};

class HealthSystem {
public:
    void update(ComponentManager& components) {
        for (EntityID entity : components.getEntitiesWith<HealthComponent>()) {
            auto& health = components.getComponent<HealthComponent>(entity);

            if (health.hp <= 0) {
                std::cout << "  Entity " << entity << " DESTROYED (HP: 0)\n";
                // In real implementation, would destroy entity here
            }
        }
    }
};

class RenderSystem {
public:
    void update(ComponentManager& components) {
        std::cout << "\n[Render]\n";
        for (EntityID entity : components.getEntitiesWith<PositionComponent>()) {
            auto& pos = components.getComponent<PositionComponent>(entity);
            std::cout << "  Entity " << entity
                      << " at (" << pos.x << ", " << pos.y << ")";

// ...continues on next slide
```

📌 2. Game with Multiple Systems (4/5)

```
// ...continued from previous slide
            if (components.hasComponent<HealthComponent>(entity)) {
                auto& hp = components.getComponent<HealthComponent>(entity);
                std::cout << " [HP: " << hp.hp << "]";
            }
            std::cout << "\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════
//  GAME LOOP
// ═══════════════════════════════════════════════════════════════

void gameLoop(ComponentManager& components,
              MovementSystem& movement,
              CollisionSystem& collision,
              HealthSystem& health,
              RenderSystem& render,
              int frames) {

    for (int frame = 1; frame <= frames; frame++) {
        std::cout << "\n========== Frame " << frame << " ==========\n";

        movement.update(components, 1.0f);
        collision.update(components);
        health.update(components);
        render.update(components);
    }
}

int main() {
    // Setup (EntityManager, ComponentManager - reused from Example 1)
    EntityManager entityManager;
    ComponentManager componentManager;

    // Create systems
    MovementSystem movementSystem;
    CollisionSystem collisionSystem;
    HealthSystem healthSystem;
    RenderSystem renderSystem;
// ...continues on next slide
```

📌 2. Game with Multiple Systems (5/5)

[Code continues across slides]

```
// ...continued from previous slide

    // Create entities
    EntityID player = entityManager.createEntity();
    EntityID enemy = entityManager.createEntity();

    // Player: Position, Velocity, Health, Collider
    componentManager.addComponent(player, PositionComponent{0, 0});
    componentManager.addComponent(player, VelocityComponent{1, 0});
    componentManager.addComponent(player, HealthComponent{100});
    componentManager.addComponent(player, ColliderComponent{1.0f});

    // Enemy: Position, Velocity, Health, Collider (moving toward player)
    componentManager.addComponent(enemy, PositionComponent{5, 0});
    componentManager.addComponent(enemy, VelocityComponent{-0.8f, 0});
    componentManager.addComponent(enemy, HealthComponent{50});
    componentManager.addComponent(enemy, ColliderComponent{1.0f});

    // Run game
    gameLoop(componentManager, movementSystem, collisionSystem,
             healthSystem, renderSystem, 5);

    return 0;
}
```

```
['========== Frame 1 ==========\n\n[Render]\n  Entity 0 at (1, 0) [HP: 100]\n  Entity 1 at (4.2, 0) [HP: 50]\n\n=======
```

topic_3_component_based_ecs

Code Examples

📌 3. Dynamic Component Addition (Runtime Composition) (1/3)

Goal: Add/remove components at runtime to change entity behavior.

Output:

Key Concepts:
- **Runtime composition**: Add/remove components dynamically
- **Temporary behaviors**: Invincibility component expires
- **Conditional logic**: Systems check for component existence

---

```
#include <iostream>
#include <vector>
#include <memory>

// (Reuse previous classes)

struct InvincibleComponent {
    float remainingTime;  // Seconds of invincibility
};

class InvincibilitySystem {
public:
    void update(ComponentManager& components, float dt) {
        for (EntityID entity : components.getEntitiesWith<InvincibleComponent>()) {
            auto& invincible = components.getComponent<InvincibleComponent>(entity);
            invincible.remainingTime -= dt;

            if (invincible.remainingTime <= 0) {
                std::cout << "  Entity " << entity << " is no longer invincible!\n";
                components.removeComponent<InvincibleComponent>(entity);
            }
        }
    }
};

class CollisionSystemWithInvincibility {
public:
    void update(ComponentManager& components) {
        auto entities = components.getEntitiesWith<PositionComponent, ColliderComponent>();

// ...continues on next slide
```

📌 3. Dynamic Component Addition (Runtime Composition) (2/3)

```
// ...continued from previous slide
        for (size_t i = 0; i < entities.size(); i++) {
            for (size_t j = i + 1; j < entities.size(); j++) {
                EntityID e1 = entities[i];
                EntityID e2 = entities[j];

                // Check collision (same as before)
                // ...

                bool collision = checkCollision(e1, e2, components);
                if (collision) {
                    // Apply damage only if NOT invincible
                    if (!components.hasComponent<InvincibleComponent>(e1) &&
                        components.hasComponent<HealthComponent>(e1)) {
                        auto& hp = components.getComponent<HealthComponent>(e1);
                        hp.hp -= 10;
                        std::cout << "  Entity " << e1 << " took damage!\n";
                    } else if (components.hasComponent<InvincibleComponent>(e1)) {
                        std::cout << "  Entity " << e1 << " is INVINCIBLE!\n";
                    }

                    // Same for e2...
                }
            }
        }
    }
};

int main() {
    EntityManager entityManager;
    ComponentManager componentManager;

    EntityID player = entityManager.createEntity();

    // Add normal components
    componentManager.addComponent(player, PositionComponent{0, 0});
    componentManager.addComponent(player, HealthComponent{100});

    std::cout << "Player created with HP: 100\n";

    // Simulate taking damage
// ...continues on next slide
```

topic_3_component_based_ecs

Code Examples

📌 3. Dynamic Component Addition (Runtime Composition) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "\nPlayer takes damage...\n";
    auto& hp = componentManager.getComponent<HealthComponent>(player);
    hp.hp -= 20;
    std::cout << "Player HP: " << hp.hp << "\n";

    // Player picks up invincibility power-up!
    std::cout << "\nPlayer picks up INVINCIBILITY power-up!\n";
    componentManager.addComponent(player, InvincibleComponent{5.0f});  // 5 seconds

    // Simulate more damage (should be blocked)
    std::cout << "\nPlayer takes damage...\n";
    if (componentManager.hasComponent<InvincibleComponent>(player)) {
        std::cout << "Damage BLOCKED! Player is invincible!\n";
    }

    // Simulate time passing
    InvincibilitySystem invincibilitySystem;
    for (int i = 0; i < 6; i++) {
        std::cout << "\nFrame " << i+1 << ":\n";
        invincibilitySystem.update(componentManager, 1.0f);
    }

    // Try damage again
    std::cout << "\nPlayer takes damage...\n";
    if (!componentManager.hasComponent<InvincibleComponent>(player)) {
        hp.hp -= 20;
        std::cout << "Damage APPLIED! Player HP: " << hp.hp << "\n";
    }

    return 0;
}
```

```
['Player created with HP: 100\n\nPlayer takes damage...\nPlayer HP: 80\n\nPlayer picks up INVINCIBILITY power-up!\n\nPl
```

topic_3_component_based_ecs

Code Examples

📌 4. Archetype-Based ECS (Cache-Friendly) (1/3)

Goal: Implement archetype storage for better cache locality.

Output:

Key Concepts:
- **Archetype storage**: Entities with same components stored together
- **Cache locality**: All positions in one contiguous array
- **Performance**: 2-5x speedup for large entity counts

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <memory>
#include <tuple>

// ═══════════════════════════════════════════════════════════════
//  ARCHETYPE (Entities with same components share storage)
// ═══════════════════════════════════════════════════════════════

template<typename... Components>
class Archetype {
public:
    std::vector<EntityID> entities;
    std::tuple<std::vector<Components>...> componentArrays;

    void addEntity(EntityID entity, Components... components) {
        entities.push_back(entity);

        // Add components to respective arrays
        size_t index = 0;
        ((std::get<index++>(componentArrays).push_back(components)), ...);
    }

    template<typename Component>
    Component& getComponent(size_t entityIndex) {
        return std::get<std::vector<Component>>(componentArrays)[entityIndex];
    }

    size_t size() const {
// ...continues on next slide
```

📌 4. Archetype-Based ECS (Cache-Friendly) (2/3)

```
// ...continued from previous slide
        return entities.size();
    }
};

// ═══════════════════════════════════════════════════════════
//  ARCHETYPE-BASED MOVEMENT SYSTEM
// ═══════════════════════════════════════════════════════════

using MovableArchetype = Archetype<PositionComponent, VelocityComponent>;

class ArchetypeMovementSystem {
public:
    void update(MovableArchetype& archetype, float dt) {
        std::cout << "\n[ArchetypeMovementSystem] Processing "
                  << archetype.size() << " entities...\n";

        // Cache-friendly! All positions in one array, all velocities in another
        auto& positions = std::get<0>(archetype.componentArrays);
        auto& velocities = std::get<1>(archetype.componentArrays);

        for (size_t i = 0; i < archetype.size(); i++) {
            positions[i].x += velocities[i].dx * dt;
            positions[i].y += velocities[i].dy * dt;

            std::cout << "  Entity " << archetype.entities[i]
                      << " at (" << positions[i].x << ", " << positions[i].y << ")\n";
        }
    }
};

int main() {
    std::cout << "═══════════════════════════════════════════════════\n";
    std::cout << "  Archetype-Based ECS (Cache-Friendly)\n";
    std::cout << "═══════════════════════════════════════════════════\n";

    MovableArchetype movableEntities;

    // Add entities with Position + Velocity
    movableEntities.addEntity(0, PositionComponent{0, 0}, VelocityComponent{1, 0});
    movableEntities.addEntity(1, PositionComponent{5, 5}, VelocityComponent{-1, 1});
// ...continues on next slide
```

topic_3_component_based_ecs

Code Examples

📌 4. Archetype-Based ECS (Cache-Friendly) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    movableEntities.addEntity(2, PositionComponent{10, 0}, VelocityComponent{0, 2});

    ArchetypeMovementSystem system;

    for (int frame = 1; frame <= 3; frame++) {
        std::cout << "\n========== Frame " << frame << " ==========";
        system.update(movableEntities, 1.0f);
    }

    std::cout << "\n\n═══════════════════════════════════════════════════\n";
    std::cout << "  BENEFIT: All Position data in one array\n";
    std::cout << "  → CPU can load multiple positions in one cache line!\n";
    std::cout << "  → 2-5x faster than scattered data\n";
    std::cout << "═══════════════════════════════════════════════════\n";

    return 0;
}
```

```
['═══════════════════════════════════════════════════\n  Archetype-Based ECS (Cache-Friendly)\n═══
```

topic_3_component_based_ecs

Code Examples

📌 5. Queued Entity Creation/Destruction (Deferred Operations) (1/4)

Goal: Handle entity creation/destruction during iteration safely.

Output:

Key Concepts:
- **Deferred operations**: Queue changes during iteration
- **Process after iteration**: Apply all changes at end of frame
- **Iterator safety**: Never invalidate iterators

```
#include <iostream>
#include <vector>
#include <queue>

// ═══════════════════════════════════════════════════════════════
//  DEFERRED ENTITY MANAGER
// ═══════════════════════════════════════════════════════════════

class DeferredEntityManager {
    EntityID nextID = 0;
    std::vector<EntityID> activeEntities;

    std::queue<EntityID> creationQueue;
    std::queue<EntityID> destructionQueue;

public:
    void queueCreateEntity() {
        EntityID newEntity = nextID++;
        creationQueue.push(newEntity);
        std::cout << "  [QUEUED] Create entity " << newEntity << "\n";
    }

    void queueDestroyEntity(EntityID entity) {
        destructionQueue.push(entity);
        std::cout << "  [QUEUED] Destroy entity " << entity << "\n";
    }

    void processQueue() {
        std::cout << "\n[Processing Queue]\n";

// ...continues on next slide
```

📌 5. Queued Entity Creation/Destruction (Deferred Operations) (2/4)

```
// ...continued from previous slide
        // Process creations
        while (!creationQueue.empty()) {
            EntityID entity = creationQueue.front();
            creationQueue.pop();

            activeEntities.push_back(entity);
            std::cout << "  CREATED entity " << entity << "\n";
        }

        // Process destructions
        while (!destructionQueue.empty()) {
            EntityID entity = destructionQueue.front();
            destructionQueue.pop();

            activeEntities.erase(
                std::remove(activeEntities.begin(), activeEntities.end(), entity),
                activeEntities.end()
            );
            std::cout << "  DESTROYED entity " << entity << "\n";
        }
    }

    const std::vector<EntityID>& getActiveEntities() const {
        return activeEntities;
    }
};

// ═══════════════════════════════════════════════════════════════
//  SPAWNER SYSTEM (Creates entities during iteration)
// ═══════════════════════════════════════════════════════════════

class SpawnerSystem {
public:
    void update(DeferredEntityManager& entityManager, int frame) {
        std::cout << "\n[SpawnerSystem] Frame " << frame << "\n";

        // Iterate over current entities
        for (EntityID entity : entityManager.getActiveEntities()) {
            std::cout << "  Processing entity " << entity << "...\n";

// ...continues on next slide
```

📌 5. Queued Entity Creation/Destruction (Deferred Operations) (3/4)

```
// ...continued from previous slide
            // Spawn new entity (queued, doesn't affect current iteration)
            if (frame == 2 && entity == 0) {
                std::cout << "    Entity 0 spawns a child!\n";
                entityManager.queueCreateEntity();
            }

            // Destroy entity (queued)
            if (frame == 3 && entity == 1) {
                std::cout << "    Entity 1 self-destructs!\n";
                entityManager.queueDestroyEntity(entity);
            }
        }
    }
};

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << "  Deferred Entity Operations\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    DeferredEntityManager entityManager;
    SpawnerSystem spawner;

    // Create initial entities
    entityManager.queueCreateEntity();
    entityManager.queueCreateEntity();
    entityManager.processQueue();

    std::cout << "\n--- Starting Simulation ---\n";

    for (int frame = 1; frame <= 4; frame++) {
        std::cout << "\n========== Frame " << frame << " ==========";

        spawner.update(entityManager, frame);
        entityManager.processQueue();

        std::cout << "\nActive entities: ";
        for (EntityID e : entityManager.getActiveEntities()) {
            std::cout << e << " ";
        }
// ...continues on next slide
```

📌 5. Queued Entity Creation/Destruction (Deferred Operations) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        std::cout << "\n";
    }

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << "  KEY BENEFIT: Iterator never invalidated!\n";
    std::cout << "  Operations queued during iteration, applied after.\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

```
['═══════════════════════════════════════════════════════\n  Deferred Entity Operations\n═══════════════════════════
```

topic_3_component_based_ecs

Code Examples

📌 6. System Dependencies and Scheduling

(Brief outline - structure only)

```
class System {
public:
    std::string name;
    std::vector<System*> dependencies;

    virtual void update() = 0;
};

class SystemScheduler {
public:
    void addSystem(System* system);
    void topologicalSort();  // Order systems by dependencies
    void update();           // Run in correct order
};

// Example usage:
movementSystem.dependencies = {};
collisionSystem.dependencies = {&movementSystem};
renderSystem.dependencies = {&movementSystem, &collisionSystem};

scheduler.addSystem(&renderSystem);  // Auto-schedules dependencies
```

topic_3_component_based_ecs

Code Examples

📌 7. Multithreaded ECS

(Brief outline - structure only)

```
class ParallelSystemScheduler {
    struct SystemInfo {
        System* system;
        std::vector<std::type_index> readComponents;
        std::vector<std::type_index> writeComponents;
    };

public:
    void scheduleParallel();  // Analyze dependencies, run safe systems in parallel
};

// Example: MovementSystem (writes Position) and RenderSystem (reads Position)
// → Can't run in parallel
//
// MovementSystem (writes Position) and AISystem (writes AI component)
// → CAN run in parallel (no overlap)
```

📌 8. Complete Game Example (Putting It All Together)

(Brief outline - comprehensive game with):
- Player, Enemies, Bullets entities
- Input, Movement, Collision, Damage, Spawn, Render systems
- Dynamic component addition (power-ups)
- Deferred entity destruction
- System execution order

What You Learned

Key Takeaways

- ✓ What is Component-Based Architecture?
- ✓ Simple Explanation: LEGO Blocks Analogy
- ✓ Why Use Component-Based Architecture?
- ✓ Problem with Deep Inheritance

→ Swipe to learn →